Introduktion till C++

Deluppgift 4.1 – Diablo: Dungeon

Uppgiftens målsättning
Den studerande ska få kunskap om och erfarenhet av att implementera ett större, mer komplext projekt i C++.

Uppgift
Bygg grunden till ett textbaserat Diablo-liknande spel.

När spelet startas ska en huvudmeny visas. Där kan man välja mellan att spela spelet och att aktivera fusk.

Spelaren ska kunna gå mellan olika rum med hjälp av dörrar.

I vissa rum finns fiender som hindrar spelaren från att gå till nästa rum. Ett enkelt stridssystem gör att spelaren kan besegra fienderna och gå vidare.

Spelet går att klara enligt ett win-condition som du själv bestämmer.

Översikt över features
Grundkraven för uppgiften är att följande fungerar:

- Spelaren ska kunna slå ihjäl monster
- Fiender ska kunna attackera spelaren (och döda denne)
- Spelaren ska kunna gå mellan rum med hjälp av dörrarna
- Fiender ska kunna hindra spelaren från att gå till nästa rum innan de dödats
- Man kan se spelarens alla egenskaper
- När spelaren dör visas ett meddelande om vad som hänt innan fönstret stängs ned.
- Spelet ska ha ett valfritt win-condition som går att uppnå samt är tydligt för spelaren.

Spelaren

Grundegenskaper och sekundära egenskaper

Spelarens ska ha *grundegenskaper* och *sekundäregenskaper*. Där de *sekundära* egenskaperna är en funktion av en eller flera *grundegenskaper*.

Minst följande *grundegenskaper* ska finnas:

- *Styrka*
- *Smidighet*
- *Fysik*

Minst följande *sekundäregenskaper* ska finnas:

- *Attackvärde* (styrka * smidighet)
- *Max-hälsa* (Fysik * 4 + styrka * 6 + smidighet * 3)
- *Bärkapacitet* (styrka + smidighet / 3)
- *Försvar* (fysik + smidighet)

Formlerna ovan är **exempel** och ni kan ändra dem om ni vill. Grundkravet är att varje *sekundäregenskap* är beroende av minst två grundegenskaper.

Visa egenskaper

Man ska via menyer i spelet kunna se spelarens alla egenskaper (och namn om ni har det). Alternativt så visas det automatiskt vid behov.

Fiender

Så länge spelaren är i ett rum där det finns fiender skall hen inte tillåtas att byta rum. Först när alla fiender är besegrade får spelaren lämna rummet. Ni är fria att bestämma hur det uppnås.

Ett exempel på lösning är att spelaren alltid har alternativet att byta rum. Men om spelaren försöker när där finns fiender så skall hen få veta att det inte går att lämna rummet så länge där är fiender som lever i rummet. Dessutom kan fienderna få en attack mot spelaren utan att spelaren får slå tillbaka.

En annan lösning kan till exempel vara att spelaren inte får andra alternativ och helt enkelt inte kan utföra andra handlingar utöver strid. Då kan alternativ att byta rum visas först när alla fiender är hanterade.

Stridssystemet

Ni har friheten att utforma stridssystemet lite som ni vill, men det finns ett par saker ni ska uppfylla:

- Attackvärde och försvar ska användas för att räkna ut skada från fiender och spelare.
- Det ska ha stöd för att spelaren ska kunna välja vem som hen attackerar. Det innebär att alla fiender attackerar spelaren, och att spelaren får välja vem som ska attackeras.
- Fiender har hälsa som minskar då spelaren lyckas med en attack mot dem. Om hälsan når noll eller blir negativ när fienden tar skada så dör fienden direkt.

Ett enkelt och konsekvent sätt att lösa det är att låta strider vara turbaserade där först spelaren attackerar och därefter fienderna. Detta upprepas tills antingen alla fiender, eller spelaren besegrats.

Rum

- Det ska finnas mellan 3 och 5 rum.
- Ett rum måste innehålla en eller flera dörrar som leder till andra rum. Det är krav på att rummen ska ha stöd för olika antal dörrar (max 4 dörrar, 1 som leder dig tillbaka till föregående rum).
- Det kan finnas fiender i rum, men alla rum behöver inte innehålla fiender.
- När spelaren kommer in i ett rum presenteras rummets namn och/eller beskrivning (så att det är lätt att hålla reda på i vilket rum man är när man rör sig mellan rum).

Dörr

- En dörr kopplar ihop två rum och används för att gå från det ena rummet till det andra. Dörrarna håller själv reda på vilka rum de leder mellan. Så när spelaren går från ett rum till ett annat måste det ske via en dörr, och dörren innehåller data om vilket rum som man kommer till.
- Man ska kunna koppla ihop vilka två rum som helst med varandra (”magiska dörrar”, till exempel en dörr som får en att gå mellan rum 1 och rum 3). Behöver inte ha en geometriskt fungerande layout
- En dörr kan vara låst eller olåst. Olåsta dörrar går att gå igenom och låsta dörrar måste öppnas innan det går att passera igenom dem.
    - Detta kan man göra genom att dyrka upp låset (smidighetscheck)
    - Elller genom att slå sönder dörren (styrkecheck).
    - Båda dessa alternativ kräver en färdighetscheck där dörrens svårighetsgrad jämförs med spelarens egenskaper för att se om en eller ett par av egenskaperna är tillräckligt hög(a) för att handlingen ska lyckas.
- Låser man upp en dörr så ska den vara upplåst åt båda håll. Verifiera detta genom att låsa upp en dörr, och sedan gå tillbaka genom samma dörr igen.

Fusk

- För att göra spelet lätt att testa ska man kunna aktivera fusk som gör spelet enklare. Olika fusk ska kunna aktiveras individuellt.
- Följande fusk ska finnas:
    - Gör spelaren odödlig
    - Gör att spelaren automatiskt besegrar fiender i ett slag

Övriga krav

- Skriv koden och kommentarer på engelska
- Allt spelaren gör ska styras med menyer. Lämpligt är att menyerna numreras 1-9 och spelaren väljer meny med siffror. Ex.:
    - [1] Attackera det stora hemska monstret
    - [2] Attackera det lilla otäcka monstret

Tillvägagångssätt Tips!

Det är en mycket stor uppgift med många olika delar. Därför kan det vara bra att först av allt strukturera uppgiften. Till exempel att hitta nyckelord och antingen göra dem **fetstilta** eller på annat sätt markera.

Börja sedan att lägga in features, **en i taget**. Punktlistan med de **objekt** ni behöver kan fungera som en lista över **ordningen ni lägger in** objekt och **punktlistan** med **features** är **sorterad efter objektlistan**, så den kan fungera som en **ordning för** att lägga in **features**. Sätt gärna egna delmål, ex:

- Det ska finnas flera rum och spelaren ska kunna gå mellan dem (mellan 3-5 rum).
- Det ska finnas fiender i rummen som man kan se. De gör ingenting ännu.
- Man kan attackera en fiende. Den dör då direkt och tas bort ur rummet.

Som ni ser måste inte delmålen innebära färdiga objekt. Det går mycket bra att börja med att objektet ”ska finnas”, som fienden i punktlistan ovan. Sedan kan man ha som mål att kunna interagera med objektet på ena hållet, ex att spelaren kan attackera fienden, men de attackerar inte tillbaka. Sedan lägger man på komplexitet efter hand.

När man gör **större uppgifter** handlar det mycket om att **bryta ner** uppgiften i så små **delmål** att det är bekvämt att arbeta med ett delmål.

Exempel *”Spelaren ska kunna slå ihjäl monster”* talar om för oss att vi behöver en spelare som kan skada monster. Det säger även att vi behöver monster som behöver ha en hälsa och att de behöver kunna ta skada. Utöver det ser vi även att det behövs en interaktion där spelarens attack producerar ett värde som subtraheras från monstrets hälsa.
Det behövs inte mer för att påbörja uppgiften, ta sedan små steg. Identifiera, implementera och verifiera delmål iterativt.

Inlämning

Gör mapp med uppgiftens namn. I denna mapp skapar ni ytterligare tvåmappar. En som heter **Exe** och en som heter **Source**.

I **Source**-mappen innehåller all källkod samt projekt filer.
Exe-mappen ska endast filer som behövs för att köra programmet ligga.
Ta bort alla skräpfiler!!

Läs igenom ”Inlämningsinstruktioner”-dokumentet för en mer komplett genomgång om hur uppgifter skall lämnas in.

Verktyg

För att lösa uppgiften är följande verktyg, eller koncept tillåtna. En del av er kanske redan har kunskaper i C++ och vet andra sätt att lösa uppgiften med hjälp av andra koncept. Det är inte tillåtet, uppgiften kan och skall lösas endast med angivna verktyg!

```
#include, #pragma once
assert
funktioner
namespace, char, short, int, long, float, double, bool, void,
unsigned
std::string, std::vector, std::bitset, iteratorer
class, struct, public, private
const
static
enum class
std::array, referenser, pekare
typedef
if, else, else if, switch, while, do, for, return, break
random_device, mt19937, uniform_int_distribution
cout, cin
static_cast<>, const_cast<>
system(“cls”), system(“pause”)
```